Image Classification with a DNN and a CNN

1 DATASET ORIGIN

The CIFAR-10 was found on the provided website: https://www.imageprocessingplace.com/root_files_V3/image_databases.htm. This dataset is also available in the Tensorflow datasets, so that was the source utilized. The dataset's homepage is https://www.cs.toronto.edu/~kriz/cifar.html

It has 10 objects or animals that act as categories for each of the 60000 images in the dataset. The dataset is balanced, offering a similar number of pictures of each of the categories.

1.1 TECHNICAL DESCRIPTION

The TensorFlow library was used to create a backpropagation Neural Network to classify 32x32-pixel color images into their categories. These networks learn by first evaluating one (online) or more (batch) input samples and checking with the input's label. The comparison with the input's label is used to evaluate whether the model is close to understanding the images and why it should classify each in one place or another. From the error calculated (loss), the model readjusts its weights using a gradient descent function.

1.2 ALGORITHM DESIGN

1.2.1 Data Preparation

The dataset was loaded from TensorFlow, already split into train and test splits. This data is already prepared, containing 32x32 pixels with RGB values. The shape of the image vectors is (60000, 32, 32, 3). 60000 images total, 32x32 pixels, 3 values per pixel. The image data values are 0-255, meaning a 8-bit per color configuration. For the NN to learn better, this is normalized to a value between 0 and 1.

1.2.2 Model Architecture

The first model built is a Convolutional NN. This is a Neural Network with convolutional and pooling layers. The global architecture is:

1. Input
2. Convolutional and MaxPool Layers: Conv1 -> MaxPool1 -> Conv2 -> MaxPool2-> Conv3
3. Flatten the resulting vector -> Dense1 -> Dense2 (output)

Each of the 3 convolutional layers look at the image in kernels of 3x3. This allows to find patterns between the neighbors of each pixel, allowing to learn about the organization of the pixels rather than only the color of each individual pixel. Having 3 allows the later ones to find more abstract patterns than the first one.

The Maxpool layers are good at giving the model the capacity to find spatial features even if they moved a bit compared to another image, by taking the maximum value on each 2x2 box. Additionally, they reduce the dimensionality to keep computation manageable, while retaining much of the information gained through the convolutional layers.

Finally, to reshape the spatial dimensions that the previous layers create into a 1D vector that can be used to classify, we flatten the final vector and pass this vector to two final Dense layers.

The first layer reduces the dimensions of the vector to a vector of 64 values, which is passed onto the 10 final probabilities. This has been found to work better than one single dense layers, because it forces the network to learn from a more compact dimension set rather than a large number of dimensions before classifying into the final 10 categories.

All the convolutional layers are activated with a ReLU activation function. This activation function is fast, and makes gradients easy to calculate in backpropagation networks. It tends to work well for a wide range of varieties of data.

The second model built is a DNN, consisiting of several Dense layers. The input is flattened for this model, since each of the values will be assigned an input neuron. Then, four layers that each reduce the number of neurons were added, with 256, 128, 64 and finally 10 neurons in the output layer. This progression allows the network to learn complex patterns because of the large number of neurons and also compress the information so a decision is made on nodes that contain multiple calculations adjusted by the training, rather than directly on barely processed input values.

Building a larger network does not improve the results, denoting an inherently bad architecture for this task rather than a model so small it cannot learn from the spatial image patterns. Not only does it not improve but it also overfits more and faster to the training data.

All the layers use a ReLU activation function, identical to the CNN.

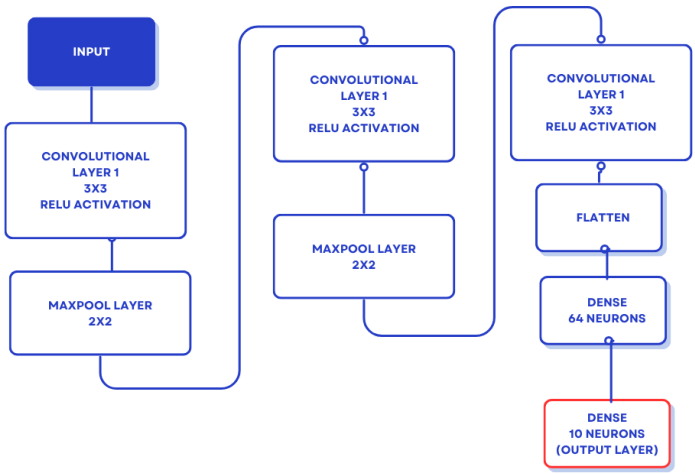
1.2.3 Configurations

The loss metric used is Categorical Crossentropy in both cases. This is a common metric used for multi-class classification, which penalizes the confident wrong answers heavily. This is positive because it allows the model to correct these large errors.

Both models have initial weights selected at random. Starting multiple times with different weights gives very similar results, showing that there is no advantage in setting specific ones. The learning was done in batches of 128 samples.

The learning rate is set to 0.0005 and the stopping criterion is stopping after 10 epochs with no fall in validation loss, and reverting to the best model after finishing the 10 extra epochs.

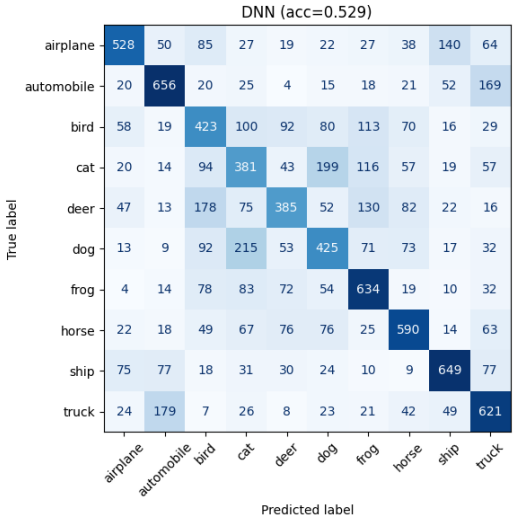
The following flowchart visually displays the layers of the convolutional NN for a structured and clear view:

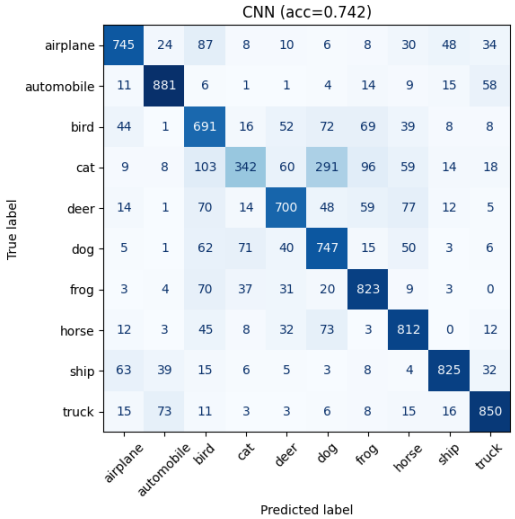


1.3 Results of the Algorithms and Analysis of the Results

Both Neural Networks achieved a performance that showed they clearly learnt from the images. However, the Convolutional NN achieved the best accuracy around 0.735, meaning 73.5% of the images were correctly classified. It was around what was expected, given that the images are 32x32 pixels, meaning some of them can be challenging to classify even for a human. However, it can surely do better because a human can recognize the images with better accuracy than the model. It is important to note that humans have seen these objects many more times and in many more situations that the neural network, meaning the model could also improve from more images of the 10 categories used.

The accuracy of the DNN was far worse, despite still doing far better than random. This can be seen in the confusion matrices below for the two described models:

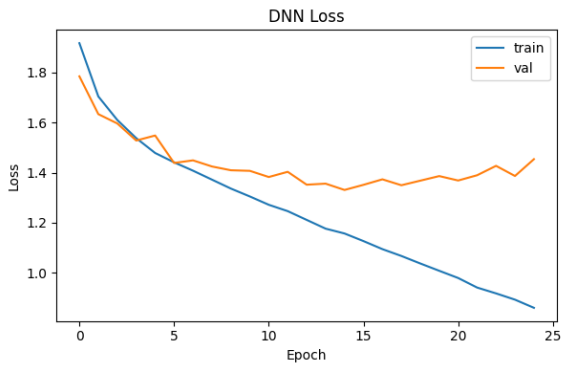


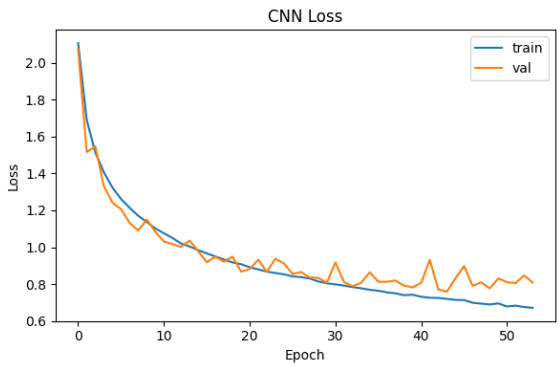


From the confusion matrix, we can see how the CNN model classifies most of the images correctly, particularly struggling in the cat, bird and dog categories. The vehicles tend to be misclassified as other vehicles, and the animals tend to be misclassified as other animals in both cases. That can be understood considering these two groups are very different from each other but similar within the groups.

The comparisons can be made clearly because there is 1000 images of each category in the test set, facilitating the visualization of common confusions, such as cat vs dog, without the need for normalization.

The following chart shows the loss over the epochs of both models. The CNN had a learning rate that maybe could have been lower, given the jumps in validation loss. However, this was the best result obtained so it was kept with that learning rate. The DNN model clearly overfits far more than the CNN, which can be seen in the difference between validation and training loss.





1.4 CODE:

```
# Load Data
import tensorflow as tf
from tensorflow.keras import layers, models, regularizers

# Check & enable GPU
gpus = tf.config.list_physical_devices('GPU')
print("GPUs available:", gpus)
for gpu in gpus:
    tf.config.experimental.set_memory_growth(gpu, True)

# Normalize data
x_train_normalized = x_train / 255.0
x_test_normalized = x_test / 255.0

x_train_flat = x_train_normalized.reshape(len(x_train_normalized), -1)
x_test_flat = x_test_normalized.reshape(len(x_test_normalized), -1)

callbacks = [tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=10, restore_best_weights=True)]

with tf.device('/GPU:0' if gpus else '/CPU:0'):
    # DNN
    dnn_model = models.Sequential([
        layers.Input(shape=(x_train_flat.shape[1],)),
        layers.Dense(256, activation='relu'),
        layers.Dense(128, activation='relu'),
        layers.Dense(64, activation='relu'),
        layers.Dense(10)
    ])
    dnn_model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
        loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        metrics=['accuracy']
    )

    # # CNN
    cnn_model = tf.keras.models.Sequential([
        tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
        tf.keras.layers.MaxPooling2D((2, 2)),
        tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
        tf.keras.layers.MaxPooling2D((2, 2)),
        tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(64, activation='relu'),
        tf.keras.layers.Dense(10)
    ])
    cnn_model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
        loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        metrics=['accuracy']
    )

    # Train
    print("\n=== Training DNN ===")
    dnn_history = dnn_model.fit(
```

```
        x_train_flat, y_train,
        epochs=100, batch_size=128,
        validation_data=(x_test_flat, y_test),
        callbacks=callbacks
    )

    print("\n=== Training CNN ===")
    cnn_history = cnn_model.fit(
        x_train_normalized, y_train,
        epochs=100, batch_size=128,
        validation_data=(x_test_normalized, y_test),
        callbacks=callbacks
    )

# TESTING BELOW
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
# first loss curve
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(dnn_history.history['loss'], label='train')
axes[0].plot(dnn_history.history['val_loss'], label='val')
axes[0].set_title('DNN Loss')
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss'); axes[0].legend()

axes[1].plot(cnn_history.history['loss'], label='train')
axes[1].plot(cnn_history.history['val_loss'], label='val')
axes[1].set_title('CNN Loss')
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Loss'); axes[1].legend()
plt.tight_layout(); plt.show()


# then metrics
dnn_loss, dnn_acc = dnn_model.evaluate(x_test_flat, y_test, verbose=0)
cnn_loss, cnn_acc = cnn_model.evaluate(x_test_normalized, y_test, verbose=0)
print(f"DNN -> loss: {dnn_loss:.4f} | accuracy: {dnn_acc:.4f}")
print(f"CNN -> loss: {cnn_loss:.4f} | accuracy: {cnn_acc:.4f}")

dnn_preds = np.argmax(dnn_model.predict(x_test_flat, verbose=0), axis=1)
cnn_preds = np.argmax(cnn_model.predict(x_test_normalized, verbose=0), axis=1)

# finally conf matrices
ConfusionMatrixDisplay(
    confusion_matrix(y_true, dnn_preds),
    display_labels=class_names
).plot(ax=axes[0], cmap='Blues', colorbar=False, xticks_rotation=45)
axes[0].set_title(f'DNN (acc={dnn_acc:.3f})')

ConfusionMatrixDisplay(
    confusion_matrix(y_true, cnn_preds),
    display_labels=class_names
).plot(ax=axes[1], cmap='Blues', colorbar=False, xticks_rotation=45)
axes[1].set_title(f'CNN (acc={cnn_acc:.3f})')
plt.tight_layout()
 plt.show()
```